Crimson Hacks 2026

Previous university projects, participant perspectives, and a practical hackathon strategy | November 6, 2026

Executive takeaway: Based on the earlier event's winners and related JustBuild competitions, prioritize a strong technical idea that can be demonstrated live and understood quickly. A finished, interactive project often communicates more than a complicated, incomplete architecture.

1. Is there an official theme?

The current Luma event listing does not publicly announce a 2026 theme or judging rubric. Crimson Hacks 2025 used the theme Mission to Mars; that does not confirm this year's theme. Confirm the official rules, allowed AI tools, code-reuse policy, team limits, and demo format at kickoff.

2. What won at Crimson Hacks 2025?

Public organizer summaries reported approximately 120 builders, 20 projects, and seven winners.

Award	Project	What made it stand out
Best Overall	Rover-Q	Q-learning rover navigation visualized on a 3D Mars landscape.
Best Technical	Marvis	Camera object detection with voice-control interaction.
Best AI/ML	Weather Aware Predictions	Image classification and forecasting connected to solar-charging needs.
Most Ambitious	Utah Robotics	Depth-camera terrain mapping and simulation with PID control.
Best Game	Catstronaut	Creative, memorable coding-themed game.
Best UX/UI	The Unemployed	Hand-drawn game design, story, and original music.
Funniest/Wildest	MarsCam	Interactive alien-themed camera filters.

The strongest common pattern is an observable result: an algorithm navigates, a model classifies, a camera responds, or a simulation adapts. Judges can understand the engineering without reading source code.

3. Participant reflections

Ishita Juluru (AI/ML winner): Highlighted learning to brainstorm and complete an end-to-end project under a short deadline. Her team's roughly five-hour build used MobileNetV2 and a small hand-analyzed Mars image set, connecting classification to predictions and energy needs.

Zach Martim (Best Overall): Described applying AI coursework in a practical project and emphasized teamwork. The lesson: a well-executed familiar algorithm can win.

Minh Le (Funniest/Wildest): Shared that this was his first in-person hackathon and first win. Coordinating on a shared concept helped turn a fun idea into a deliverable.

Andrew Radford (organizer): Highlighted creative projects, challenging problems, practical engineering, and mentorship opportunities.

These are first-hand public reflections, not formal judging reports or detailed technical postmortems.

4. Broader organizer patterns

Other JustBuild-associated events featured AI appointment-scheduling agents, phone-controlling agents, customer-simulation agents, an AI on-call engineer, real-time video recognition, and AI roofing analysis. These are not previous Crimson Hacks winners, but they suggest the organizers' ecosystem values useful autonomous behavior, interesting technical work, and visible demonstrations.

5. Recommended competitive strategy

One difficult problem + one visible technical insight + one excellent demo. Choose the award category after the rubric is announced. A good primary target is Best Overall, with Best Technical or Best AI/ML as a possible secondary fit.

Factor	Suggested weight	Question to ask
Technical substance	25%	Is there a real algorithmic or engineering challenge?
Demo impact	25%	Can judges change an input and see a result?
Theme fit	20%	Does it solve the announced problem?
Finishability	20%	Can the full path work before demos?
Originality	10%	Is the interpretation memorable?

Weights are my preparation heuristic, not the official judging rubric.

6. Adaptable project directions

A. Interactive optimization simulator (top pick): Show a route, allocation, scheduling, or control algorithm operating under changing constraints. Possible techniques: A*, Q-learning, constraint optimization. Let judges alter the environment and see the recomputed result.

B. AI agent that takes real actions: Build a narrow, end-to-end workflow with visible tool calls and verifiable results. Avoid a generic chat UI and avoid elaborate multi-agent infrastructure.

C. Real-time perception + decision: Use a webcam, audio, or sensor input for classification or tracking, then generate an observable action. Great demo potential, but account for device reliability and model latency.

7. Five-hour build timeline

Time	Objective
0-30 minutes	Read theme and rules; commit to idea, demo, and team roles.
30-90 minutes	Prove the riskiest technical element.
90-180 minutes	Connect the core engine to a usable interface.
180-240 minutes	Polish interaction, feedback, and key visuals.
240-300 minutes	Test, record fallback demo, rehearse pitch.

If you have teammates, assign core logic, UI, and integration/testing early. Agree on interfaces before writing parallel code.

8. A 90-second judging demo

0-15 seconds: Define the problem. 15-50: Demonstrate the working product, including a live change of input. 50-75: Explain the difficult technical element and one design choice. 75-90: Show the result and describe the next improvement.

9. Preparation checklist

Check your local development tools, API keys, and deployment path. Review a few flexible algorithms without prebuilding submission code. Find teammates with complementary skills. Ask organizers what the judging emphasizes and whether the 2025 no-prewritten-code rule applies. Keep a backup screen recording.

Sources and further reading

2026 event page

JustBuild events and winners

JustBuild community

Ishita Juluru's project reflection

Minh Le's reflection

Zach Martim profile

Historical results and public accounts inform this strategy but cannot predict the 2026 theme or judging decisions.